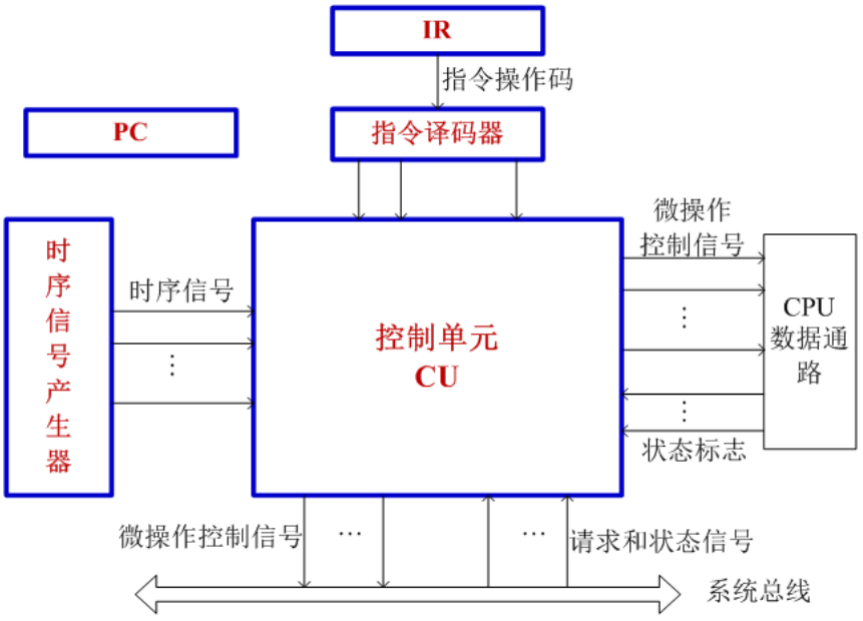
控制器

控制器基本结构和设计方法

控制器的作用：向计算机中的各个部件提供它们协调运行时所需要的控制信号
定时信号：基本定时信号为主时钟信号，进行分频处理可产生其它不同作用的定时信号
控制信号：指令周期被分为多阶段，每个阶段执行一个微操作序列完成不同功能，控制器发出完成操作所需控制信号
响应中断请求：暂停现行程序执行，进入中断周期，中断结束后，恢复原程序

控制器由三个基本部分构成：

- 控制寄存器和译码器：PC、IR、ID等
- 时序信号产生器
- 控制单元CU：核心



控制器设计核心是控制单元的设计：

- 组合逻辑设计方法/硬布线设计方法：控制单元是组合逻辑网络，产生所有控制信号，比较原始
- 微程序设计方法/存储逻辑设计方法：控制单元称为微程序控制单元，核心是控制存储器CM，由微程序解释执行机器指令，产生指令运行所需要的全部控制信号
- 组合逻辑与微程序混合设计方法：简单指令用组合逻辑，复杂指令用微程序。

计算机的控制方法

时序系统：由一组作为时间基准的信号（时序信号）组成

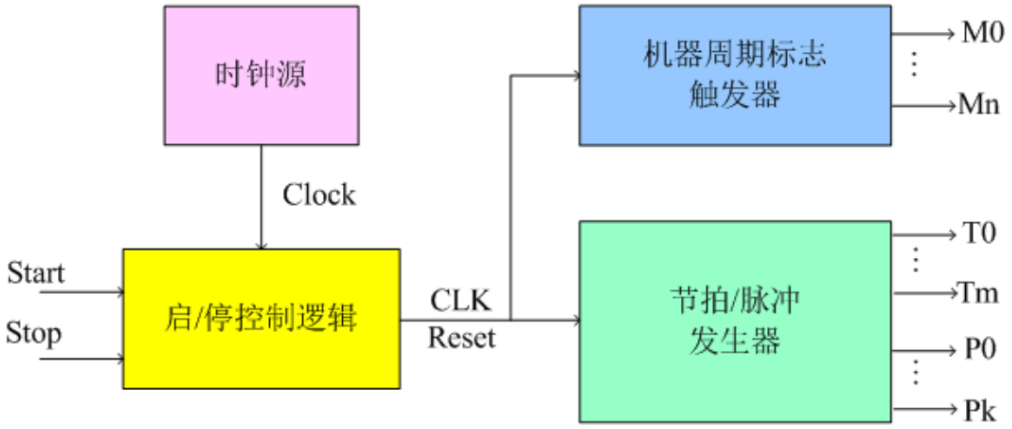
时序信号基本控制：电位-脉冲工作方式

三级时序体制：

- 主状态周期：机器周期、CPU周期
- 节拍电位：一个机器周期内执行多个相互独立的操作，分为节拍电位。一个节拍对应一个时钟周期，控制电位型信号

- 节拍脉冲：一个节拍电位中包含多个节拍脉冲，产生脉冲型信号，作为寄存器或触发器的控制信号

时序产生器为全机提供全部时序信号



控制方式：控制不同操作序列时序的方法，分为同步控制和异步控制

指令执行过程中的每一步就是一次线路状态的转换，结果存放于寄存器或存储器

同步控制方式：采用统一的时间基准信号为所有操作进行定时。特点：以**最慢**部件操作时间为依据进行时序设计；每个操作在确定的时间启动和结束，操作之间自动衔接，简单、效率低。采用三级时序系统

- 完全统一的机器周期和节拍
- 不定长机器周期
- 中央控制与局部控制结合

异步控制方式：不需要统一的时间基准信号来定时

- 每条指令、每个操作需要多久就占用多少时间
- 发出控制信号后等待执行部件完成操作后送回完成信号，再开始新的操作
- 充分发挥各部件的运行速度，时间利用率高，控制复杂
- 采用二级时序系统：节拍、脉冲，没有机器周期

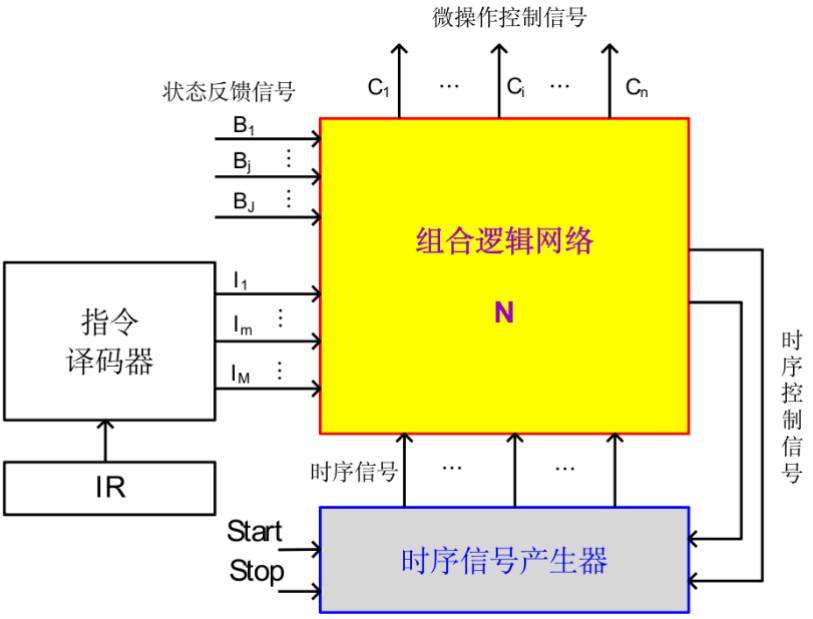
联合控制方式：同步异步相结合

- 大部分操作固定机器周期，某些异步
- 根据倾向性分为同步计算机和异步计算机，同步以同步为主，异步以异步为主
- CPU内部采用同步，主机、IO间用异步，CPU、内存间用同步或异步

组合逻辑控制器

硬布线控制器：由产生固定时序信号的逻辑电路和产生控制命令的组合逻辑网络实现，可看成是由门电路和触发器构成的复杂树形网络

设计方法：对系统进行总体设计；设计控制器结构和时序系统；设计控制单元



控制单元设计：

- 在数据通路上排出每条指令的指令周期流程，把每一步操作分解为微操作序列
- 为每一个微操作分配时间，列出微操作对应的微命令时间表
- 写出微命令的最简逻辑表达式：$C_i = f(Im, Bj, Ml, Tk, Pn)$
    - Im：指令译码信号
    - Bj：机器状态
    - Ml：机器周期
    - Tk：节拍
    - Pn：脉冲
- 画出产生全部微命令的逻辑图
- 用电路实现控制单元

主控制单元功能描述：单周期时序简化为主时钟控制

❍主控制单元功能表

输入/输出	信号名	R型	lw	sw	beq	j
输入	Op5	0	1	1	0	0
	Op4	0	0	0	0	0
	Op3	0	0	1	0	0
	Op2	0	0	0	1	0
	Op1	0	1	1	0	1
	Op0	0	1	1	0	0
输出	RegDst	1	0	X	X	X
	ALUSrc	0	1	1	0	X
	MemtoReg	0	1	X	X	X
	RegWr	1	1	0	0	0
	MemWr	0	0	1	0	0
	MemRd	0	1	0	0	0
	Branch	0	0	0	1	0
	Jump	0	0	0	0	1
	ALUOp1	1	0	0	0	X
	ALUOp0	0	0	0	1	X

❍主控制单元控制信号逻辑表达式

$$RegDst = \overline{Op_0} \cdot \overline{Op_1} \cdot \overline{Op_2} \cdot \overline{Op_3} \cdot \overline{Op_4} \cdot \overline{Op_5}$$

$$ALUScr = Op_0 \cdot Op_1 \cdot \overline{Op_2} \cdot \overline{Op_3} \cdot \overline{Op_4} \cdot Op_5 + Op_0 \cdot Op_1 \cdot \overline{Op_2} \cdot Op_3 \cdot \overline{Op_4} \cdot Op_5$$

$$MemtoReg = Op_0 \cdot Op_1 \cdot \overline{Op_2} \cdot \overline{Op_3} \cdot \overline{Op_4} \cdot Op_5$$

$$RegWr = \overline{Op_0} \cdot \overline{Op_1} \cdot \overline{Op_2} \cdot \overline{Op_3} \cdot \overline{Op_4} \cdot \overline{Op_5} + Op_0 \cdot Op_1 \cdot \overline{Op_2} \cdot \overline{Op_3} \cdot \overline{Op_4} \cdot Op_5$$

$$MemWr = Op_0 \cdot Op_1 \cdot \overline{Op_2} \cdot Op_3 \cdot \overline{Op_4} \cdot Op_5$$

$$MemRd = Op_0 \cdot Op_1 \cdot \overline{Op_2} \cdot \overline{Op_3} \cdot \overline{Op_4} \cdot Op_5$$

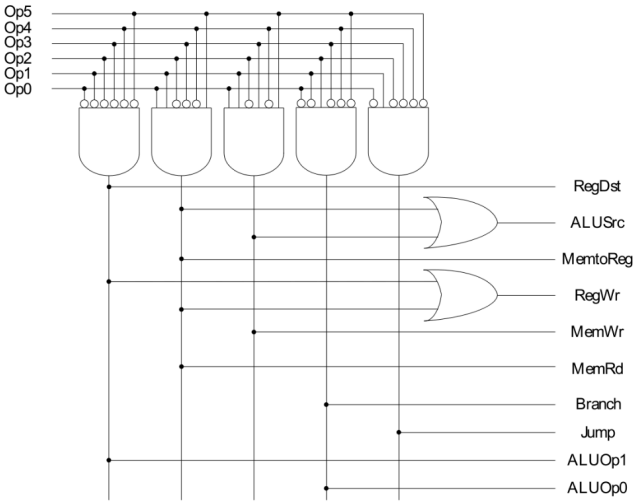
$$Branch = \overline{Op_0} \cdot \overline{Op_1} \cdot Op_2 \cdot \overline{Op_3} \cdot \overline{Op_4} \cdot \overline{Op_5}$$

$$Jump = \overline{Op_0} \cdot Op_1 \cdot \overline{Op_2} \cdot \overline{Op_3} \cdot \overline{Op_4} \cdot \overline{Op_5}$$

$$ALUOp_1 = \overline{Op_0} \cdot \overline{Op_1} \cdot \overline{Op_2} \cdot \overline{Op_3} \cdot \overline{Op_4} \cdot \overline{Op_5}$$

$$ALUOp_0 = \overline{Op_0} \cdot \overline{Op_1} \cdot Op_2 \cdot \overline{Op_3} \cdot \overline{Op_4} \cdot \overline{Op_5}$$

❍主控制单元逻辑电路



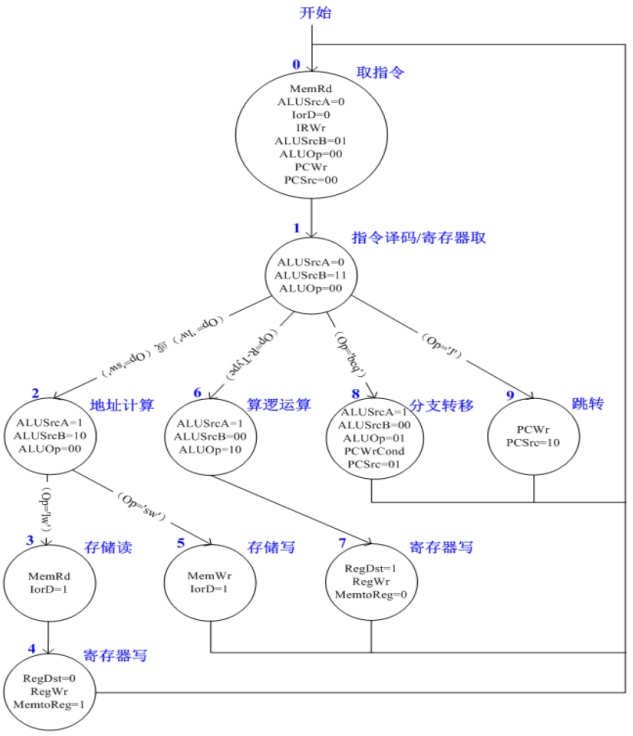
门阵列控制器：由门阵列器件（可编程逻辑器件）设计的控制器

设计方法：类似硬布线控制器，用门阵列芯片实现控制逻辑

多周期同步控制单元设计：

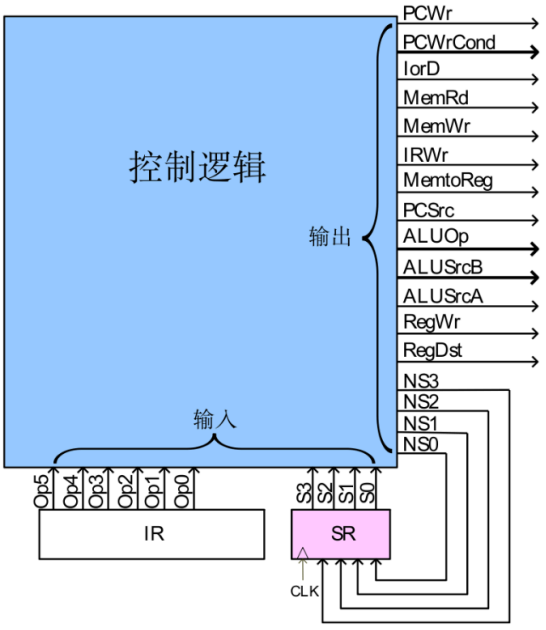
❒ 同步控制状态转换图

可以用一个有限状态机（FSM）描述多周期CPU控制单元。控制信号的不同组合值对应于有限个状态，分别用数字编号表示；由主时钟信号CLK触发状态改变。

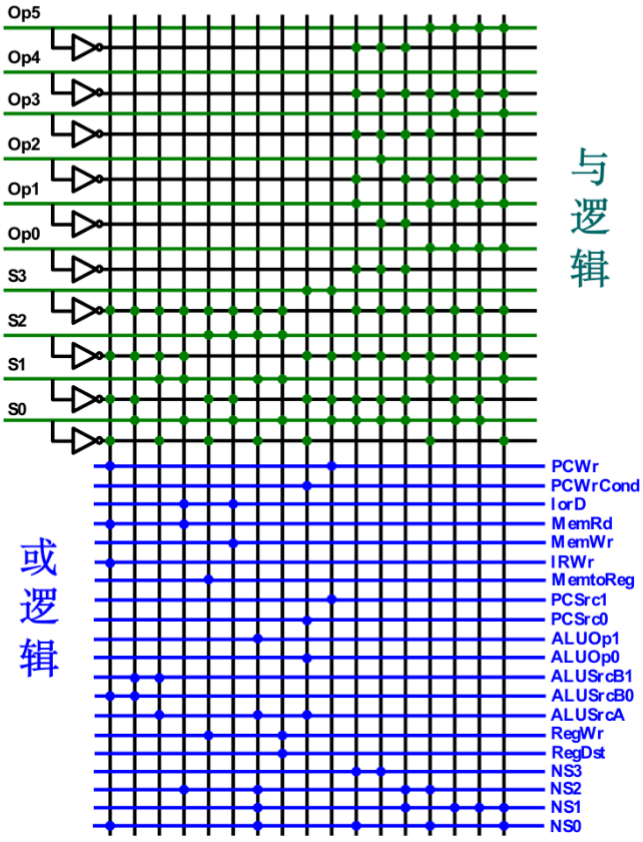


❒ 同步控制单元框图

控制单元由一个组合逻辑模块和一个保存当前状态的寄存器（SR）组成。



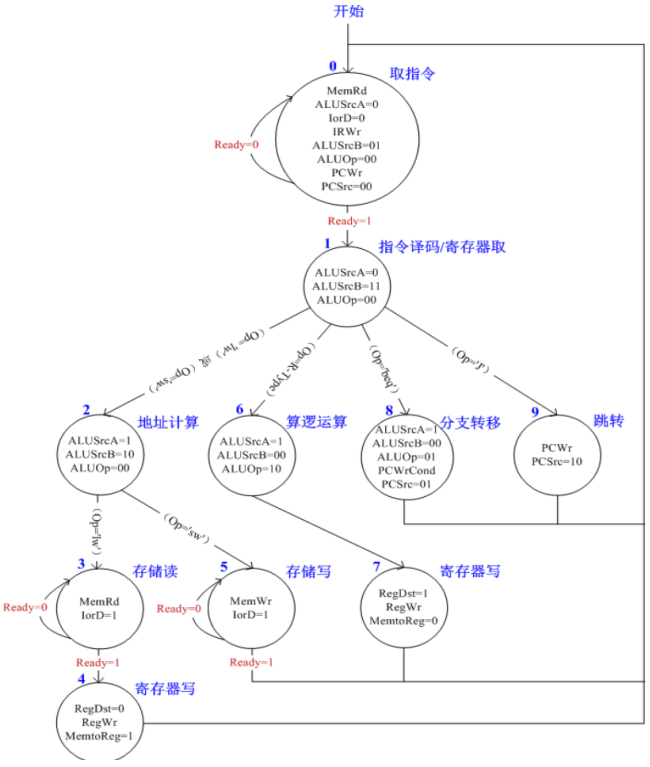
❒ 用PLA实现的控制单元



多周期异步控制单元设计：

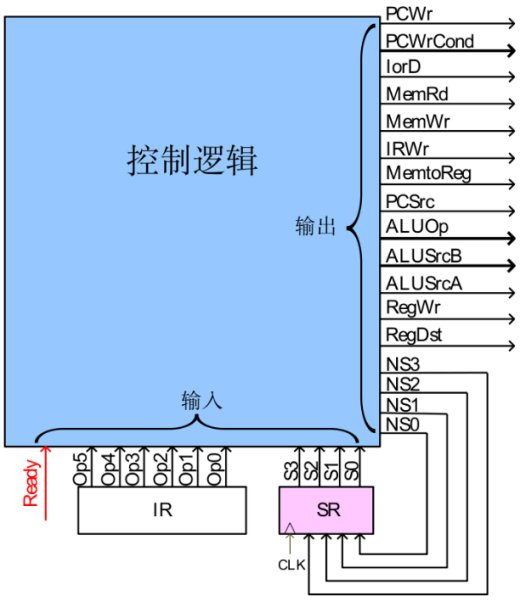
❒ 异步控制状态转换图

假设存储器和CPU之间按照异步方式工作。CPU向存储器发送MemRd或MemWr控制命令，存储器向CPU发送Ready信号作为应答。



❒ 异步控制单元框图

控制单元由一个组合逻辑模块和一个保存当前状态的寄存器（SR）组成。



特点：简单，繁琐，难以修改，速度高

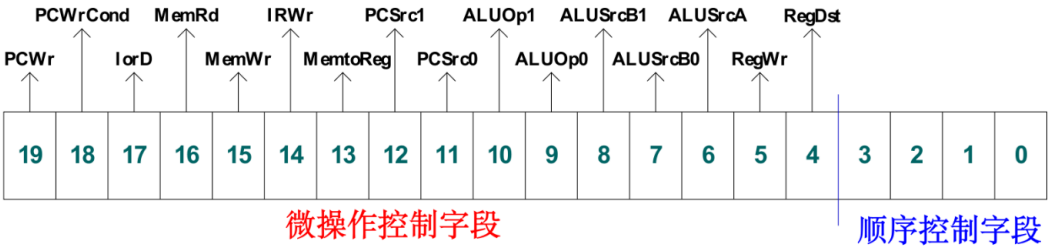
微程序控制器

微命令01表示，一堆微命令合成一个微指令，一个机器指令用若干微指令实现为微程序，微操作是微指令+类PC

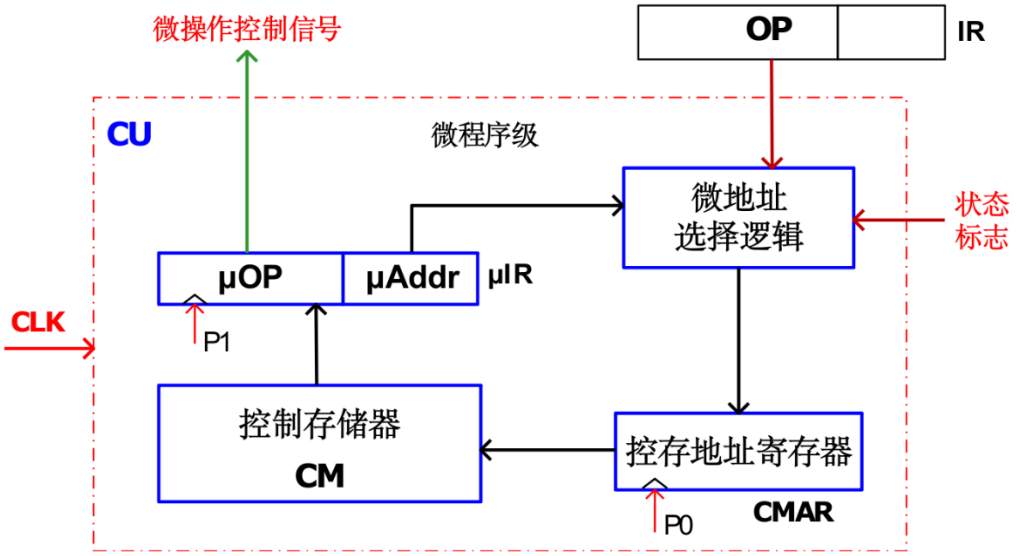
微程序设计思想：将**微命令**二进制代码化（1表示有命令，0表示无命令），将一段时间内所有微命令组织在一起形成**微指令**，将一条机器指令所需要的全部微命令用若干微指令实现，称为**微程序**，将所有微程序存储在一个特定存储器中称为**控制存储器（CM）**，通过执行微程序实现全部控制功能

微指令由**微操作控制字段**（μOP）和**顺序控制字段**（$\mu Addr$）组成：
微操作：指出微指令操作性质，即由该字段能产生全部相应微操作控制信号（微命令）
顺序：指出下一跳微指令地址

❒ 微指令格式举例



❒ 微程序控制单元基本组成



基本组成：

- 控制存储器CM（微程序存储器μPM）：存放实现全部指令系统对应的微程序
- 微指令寄存器μIR：存放由控制存储器读出的一条微指令
- 微地址选择逻辑：形成下一条微指令地址
- 微地址寄存器μAR（控存地址寄存器CMAR）：暂存将要访问的下一条微指令在控存中的地址

微指令设计基本原则：缩短长度，减小控存容量，提高速度，可以修改，提高灵活性

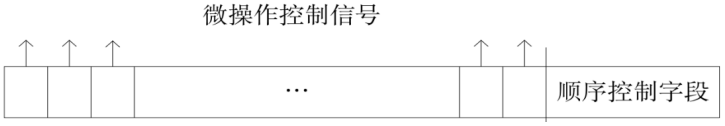
水平型微指令：在一条微指令中定义执行多个微操作的微命令。并行、微指令长微程序的短，难用

垂直型微指令：在一条微指令中仅提供一两种微命令，格式与指令相似。并行弱，微指令短微程序长，易使用

微指令格式设计：（水平型）

微命令编码：

- 直接编码（直接控制、不译法）：简单直观，速度快，微指令字长长，适用于简单机器
  微操作控制字段的每一位表示一个微命令，“0”表示无效（一种状态），“1”表示有效（另一种状态）。

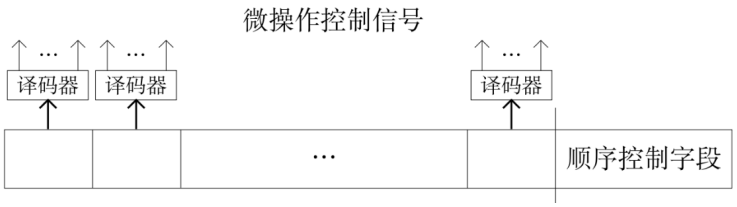


  - 相容性微命令：能在同时或同一个时间段内有效的微命令
  - 互斥性微命令：不能再同时或同一个时间段有效的微命令

- 字段直接编码方式（分段直接编译法）：

  将微操作控制字段分为若干小字段，把一组互斥微命令组织在一起，用一个小字段编码表示，将相容的微命令安排在不同字段内。在某一时间段，每个字段通过译码产生一条微命令，不同字段可以发出多条微命令。

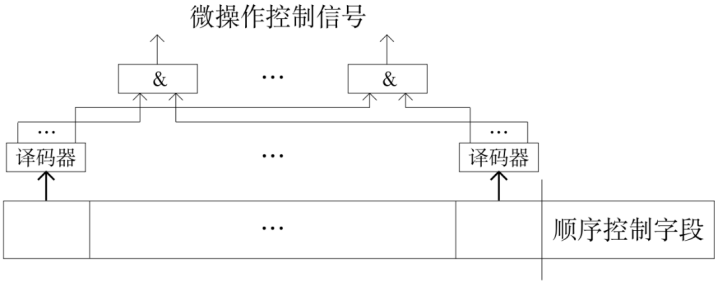
  特点：能有效压缩微指令的长度，但由于译码稍影响速度。



- 字段间接编码方式：

  在字段直接编码方式的基础上，若规定一个字段的某些微命令，要兼由另一个字段中的某些微命令来解释，称为字段间接编码方式。

  特点：在字段直接编码法的基础上，进一步缩短了微指令的长度，但可能消减微指令的并行控制能力。通常只作为直接编码法的一种辅助手段。



- 混合编码方式：分段编码方式中小字段小到一位，认为与直接编码方式混合

❍微命令编码方式举例

某机的**微指令格式**中，共有**8个**控制字段，每个字段可分别激活**5、8、3、16、1、7、25、4**种控制信号。分别采用**直接编码**和**字段直接编码**方式设计微指令的操作控制字段，并说明两种方式的操作控制字段**各取几位**。

解：

(1) 采用直接编码方式时，直接用微指令的操作控制字段的一位表示1个微命令，则操作控制字段的位数等于微命令个数，即

总位数 = 5+8+3+16+1+7+25+4 = 69位

(2) 采用字段直接编码方式时，每个小字段中的微命令应为互斥的。此时为每个小字段分配位数时要在有效微命令个数的基础上再加1种无操作状态，则每个小字段需给出的状态数分别为6、9、4、17、2、8、26、5种，据此设计的微指令的操作控制字段格式如下：



共分8个小字段，微指令操作控制字段的总位数为：

总位数=3+4+2+5+1+3+5+3=26位

与直接编码法相比，压缩了：69-26 = 43位

可见，字段直接编码法可有效地缩短微指令字长。

注意：字段直接编码中给每个字段加1个无操作状态

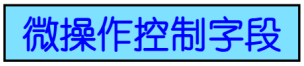
微地址形成：

- 直接表示方式：顺序控制字段直接给出后继地址



特点：简单，但微指令字长较长。

- 增量方式（计数器方式）：硬件上CMAR增加计数功能成为μPC，指令分为顺序微指令和转移微指令



（a）顺序型微指令



（b）转移型微指令

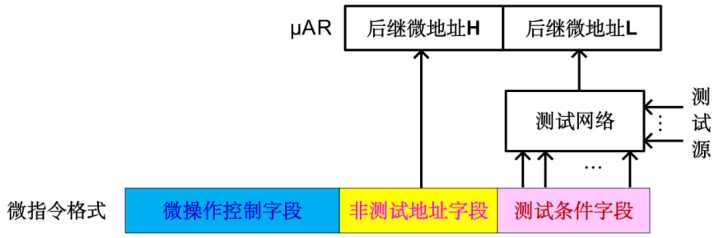
- 特点：微指令字长较短，但转移灵活性差

- 增量与下址字段相结合方式：硬件上CMAR担当μPC，仅设一种微指令，每条微指令都能转移（**只能实现两路分支转移**）



- 特点：指令格式统一，但微指令字长较长

- 断定方式：后继微地址只能由微指令顺序控制字段产生，每一条微指令都能转移，CMAR不能计数；微指令中给出后继微地址部分位，其余由机器运行状态和测试条件来断定



- **特点：可实现微程序的多路分支转移，转移路数由微地址被断定的位数决定。**

功能转移：根据特定机器指令进入对应微程序执行，按照机器指令操作码转移到相应微程序入口。功能转移属于微程序多路分支，需要外加相应硬件逻辑：硬件查表法、转移地址产生逻辑

以MIPS多周期（分散互连）控制单元设计为例，采用水平型微指令结构，介绍微指令格式和微程序的基本设计方法。

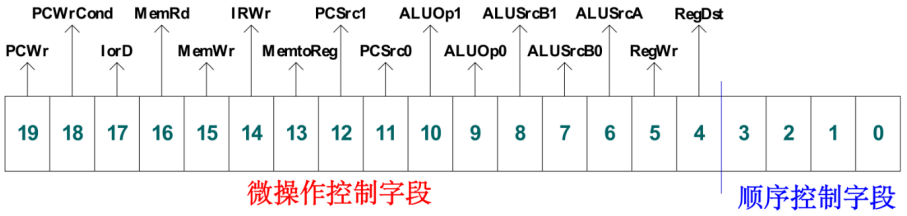
❒ 设计实例一

❍微命令编码方式：直接编码方式

❍微地址形成方法：直接表示方式

❍微入口：二级功能转移（指令操作码译码，LW/SW）

❍微指令格式



❍微程序

微指令名称	微指令地址（二进制）	微指令（二进制代码）																		
		操作控制															顺序控制			
		PCWr			MemRd			IRWr							ALUSrcB0					
		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Fetch1 Fetch2	0000	1	0	0	1	0	1	0	0	0	0	0	0	1	0	0	0
	0001	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	X	X	X	X
LW/SW	0010	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	Y	Y	Y	Y
LW1 LW2	0011	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0100	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0
SW	0101	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
R-type1 R-type2	0110	0	0	1	1	0	0	0	0	0	1	0	0	0	1	0	0	0	1	1	1
	0111	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0
BEQ	1000	0	1	0	0	0	0	0	0	1	0	1	0	0	1	0	0	0	0	0	0
JUMP	1001	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0

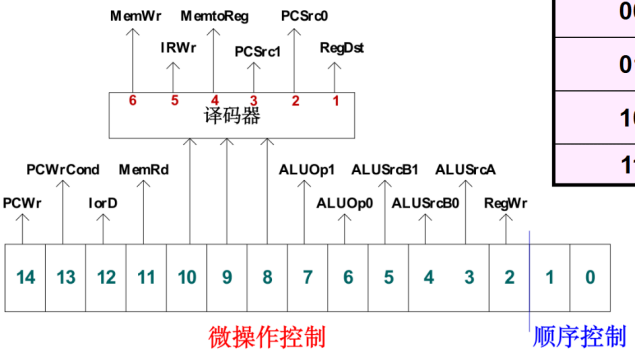
功能转移1

功能转移2

❒ 设计实例二

- ❍微命令编码方式：混合编码方式
- ❍微地址形成方法：编码选择微地址
- ❍微指令格式

顺序控制	微地址操作
00	转取指令（0号单元）
01	功能转移1
10	功能转移2
11	顺序执行（+1）



❍微程序

回上页 转下页

微指令名称	微指令地址（二进制）	微指令（二进制代码）																			
		操作控制																顺序控制			
		19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Fetch1 Fetch2	0000	1	0	0	1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	1
	0001	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	X	X	X	X
LW/SW	0010	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	Y	Y	Y	Y
LW1 LW2	0011	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
	0100	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0
SW	0101	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
R-type1 R-type2	0110	0	0	1	1	0	0	0	0	0	1	0	0	0	1	0	0	0	1	1	1
	0111	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0
BEQ	1000	0	1	0	0	0	0	0	0	1	0	1	0	0	1	0	0	0	0	0	0
JUMP	1001	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0

❍微程序

微程序名称	微指令地址（二进制）	微指令（二进制代码）															
		操作控制														顺序控制	
		14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Fetch1 Fetch2	0000	1	0	0	1	1	0	1	0	0					1	1	
	0001	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1	
LW/SW	0010	0	0	0	0	0	0	0	0	0					1	0	
LW1 LW2	0011	0	0	1	1	0	0	0	0	0	0	0	0	0	1	1	
	0100	0	0	0	0	1	0	0	0	0					0	0	
SW	0101	0	0	1	0	1	1	0	0	0					0	0	
R-type1 R-type2	0110	0	0	1	1	0	0	0	1	0	0	0	1	0	1	1	
	0111	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	
BEQ	1000	0	1	0	0	0	1	0	1	1	0	0	1	0	0	0	
JUMP	1001	1	0	0	0	0	1	1	0	0	0	0	0	0	0	0	

功能转移1
功能转移2
转取指令
+1

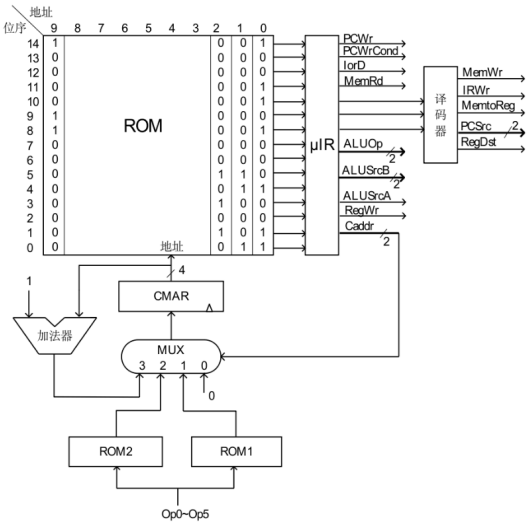
❍微程序控制单元框图

ROM1（功能转移1）

指令操作码（Op5~Op0）	指令名称	后继微地址
000000	R-type	0110
000010	jump	1001
000100	beq	1000
100011	lw	0010
101011	sw	0010

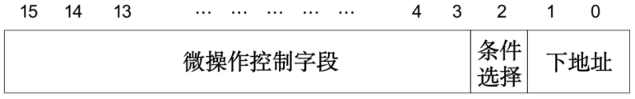
ROM2（功能转移2）

指令操作码（Op5~Op0）	指令名称	后继微地址
100011	lw	0011
101011	sw	0101



❒ 设计实例三

- ❍采用异步内存方式：存储器以Ready应答 MemRd、MemWr命令
- ❍微命令编码方式：混合编码方式
- ❍微地址形成方法：条件选择+编码选择微地址
- ❍微指令格式



条件选择	操作	Ready	微地址
0	不测试		下址字段
1	测Ready	0	保持不变
		1	下址字段

❍微程序

微程序名称	微指令地址（二进制）	微指令（二进制代码）															
		微操作码													条件	下地址	
		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Fetch1 Fetch2	0000	1	0	0	1	1	0	1	0	0	0	1	0	0	1	1	1
	0001	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
LW/SW	0010	0	0	0	0	0	0	0	0	0	1	0	1	0	0	1	0
LW1 LW2	0011	0	0	1	1	0	0	0	0	0	0	0	0	0	1	1	1
	0100	0	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0
SW	0101	0	0	1	0	1	1	0	0	0	0	0	0	0	1	0	0
R-type1 R-type2	0110	0	0	1	1	0	0	0	1	0	0	0	1	0	0	1	1
	0111	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0
BEQ	1000	0	1	0	0	0	1	0	1	1	0	0	1	0	0	0	0
JUMP	1001	1	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0

测Ready

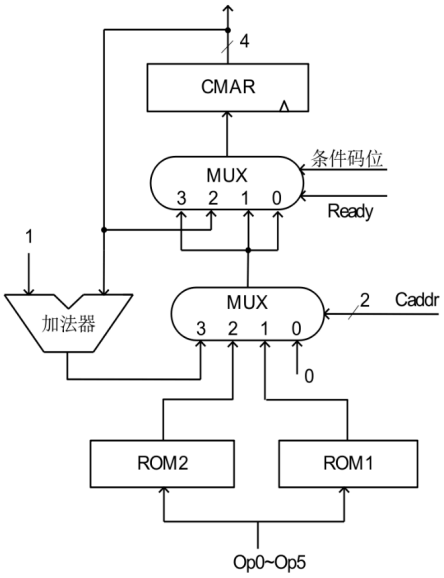
❍微地址选择逻辑

ROM1

指令操作码(Op5~Op0)	指令名称	后继微地址
000000	R-type	0110
000010	jump	1001
000100	beq	1000
100011	lw	0010
101011	sw	0010

ROM2

指令操作码(Op5~Op0)	指令名称	后继微地址
100011	lw	0011
101011	sw	0101



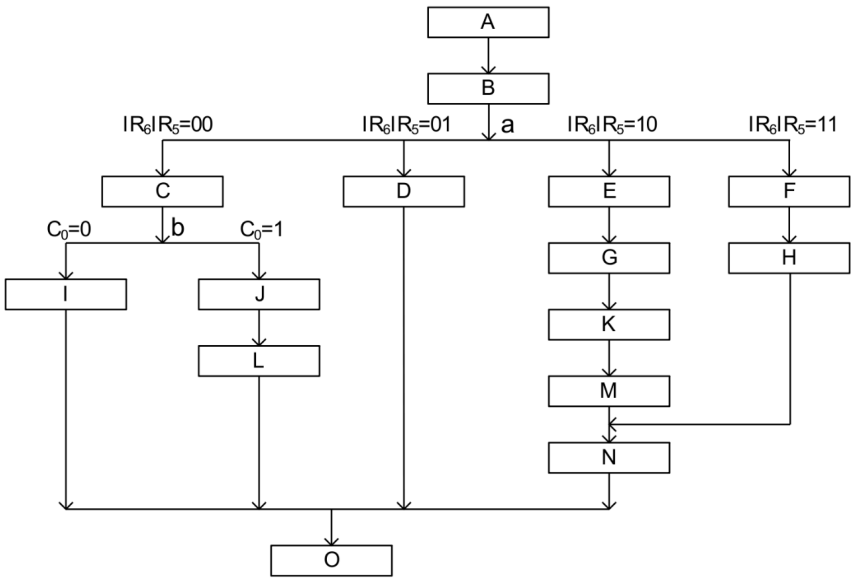
例题:

- 下图给出了某微程序控制计算机的部分微指令序列，图中每一框代表一条微指令。分支点a由指令寄存器IR的第5、6两位决定，分支点b由条件码C_0决定。已知微指令地址寄存器长度为8位。请采用增量与下址字段相结合方式实现微程序的顺序控制。具体要求如下：

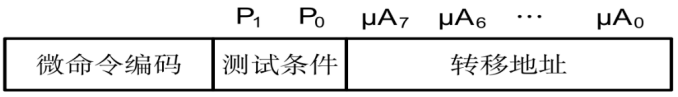
(1) 设计实现该微指令序列的微指令顺序控制字段格式；

(2) 给出每条微指令的二进制编码地址；

(3) 设计微地址转移逻辑。



解：（1）该微指令顺序控制字段格式如下：



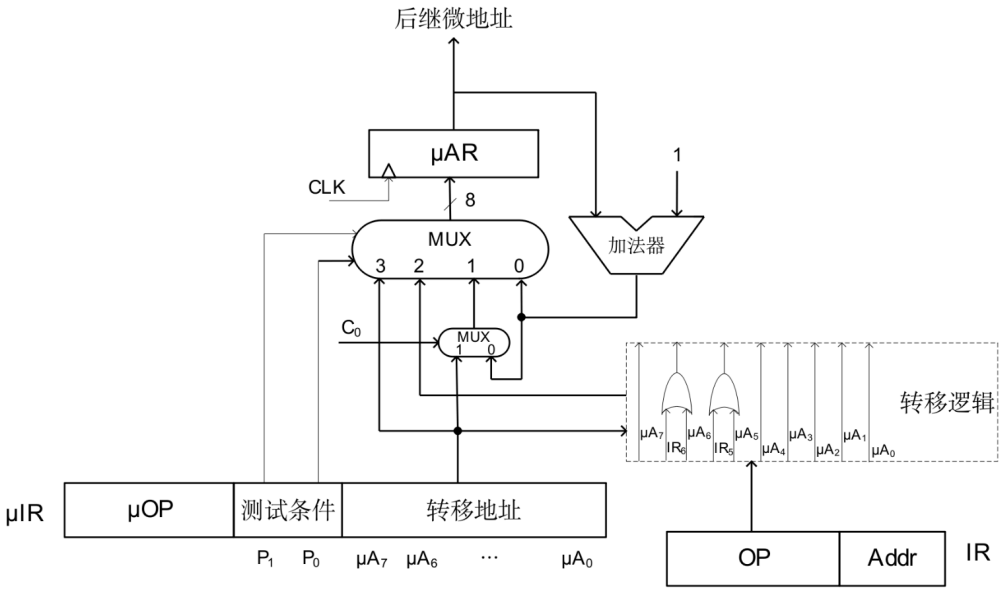
- P_1P_0=00，表示顺序执行；
- P_1P_0=01，表示对C_0进行测试，C_0=0，顺序执行，C_0=1，按转移地址跳转；
- P_1P_0=10，表示按照机器指令码中IR_5、IR_6的值实现功能转移；
- P_1P_0=11，表示按照转移地址无条件转移。

微指令	微指令地址	微指令编码									
		μOP	P_1	P_0	μA_7	μA_6	μA_5	μA_4	μA_3	μA_2	μA_1	μA_0
A	00000000	—	0	0	X	X	X	X	X	X	X	X
B	00000001	—	1	0	1	0	0	0	0	0	0	0
C	10000000	—	0	1	1	0	0	0	0	0	1	0
D	10100000	—	1	1	0	0	0	0	0	0	1	1
E	11000000	—	0	0	X	X	X	X	X	X	X	X
F	11100000	—	0	0	X	X	X	X	X	X	X	X
G	11000001	—	0	0	X	X	X	X	X	X	X	X
H	11100001	—	1	1	1	1	0	0	0	1	0	0
I	10000001	—	1	1	0	0	0	0	0	0	1	1
J	10000010	—	0	0	X	X	X	X	X	X	X	X
K	11000010	—	0	0	X	X	X	X	X	X	X	X
L	10000011	—	1	1	0	0	0	0	0	0	1	1
M	11000011	—	0	0	X	X	X	X	X	X	X	X
N	11000100	—	1	1	0	0	0	0	0	0	1	1
O	00000011	—	—		—							

注：“—”表示不是该题目要求设计的内容，“X”表示其值任意的1位二进制代码。

（3）后继微地址设计方案

- 分支a处是功能转移，需要由微指令顺序控制字段中给出的转移地址和转移逻辑共同产生后继微地址。一种可能的实现方案是：采用指令操作码相应位IR_6、IR_5直接作为后继微地址位μAR_6、μAR_5的取值。则分支a处的地址转移逻辑表达式为：
  - $\mu AR_6=IR_6$
  - $\mu AR_5=IR_5$
- 分支b处是两路分支转移，C_0=0时顺序执行，C_0=1时由转移地址字段直接给出后继微地址。
- 顺序执行时，后继微地址由μAR+1（μPC+1）产生。
- 后继微地址形成逻辑框图如下：



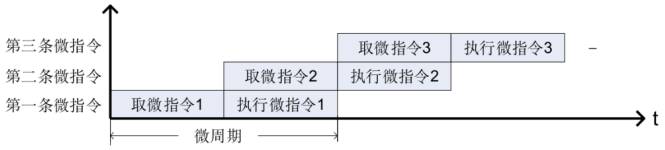
微指令周期(微周期)：取微指令、执行微指令（发出微命令）

微操作定时方式：串行方式/重叠方式（执行本条、取下一条一起）

❒ 两种微指令执行方式基本时空关系

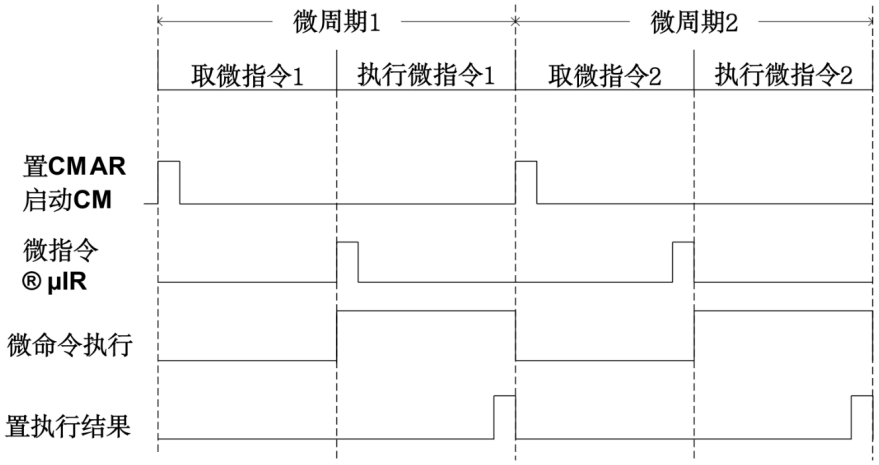


（a）串行方式

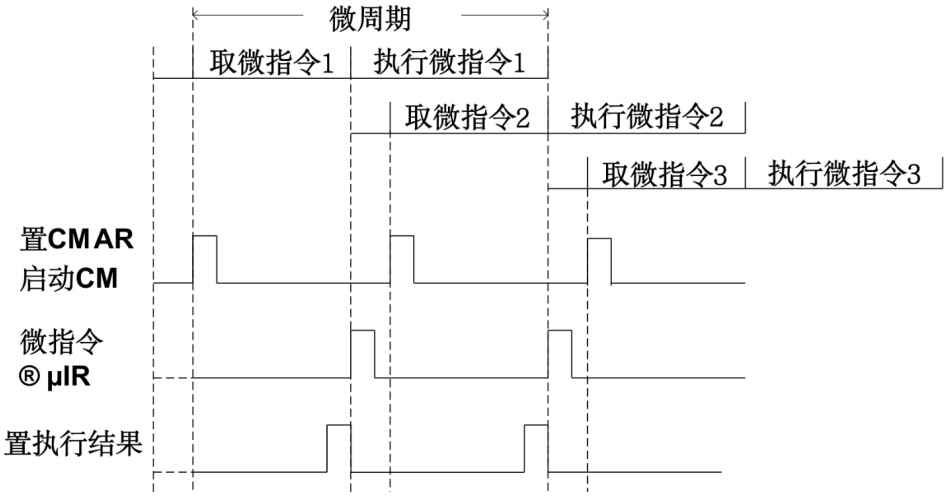


（b）重叠方式

❒ 串行微指令操作时序（三相脉冲）



❒ 重叠微指令操作时序（三相脉冲）



流水线控制器

在没一个流水步骤设定相应控制信号的值。

- FI：读指令、修正PC
- ID：指令译码，读RF
- EX：指令执行、地址计算
- MA：数据读写、转移判断
- WB：数据写回寄存器

❍ 五段流水线中第一（FI）和第二段（ID）控制信号完全相同；

❍ 将第三（EX）、第四（MA）、第五（WB）流水段需要的控制信号分成三组。

指令	EX段控制信号				MA段控制信号				WB段控制信号	
	RegDst	ALUOp1	ALUOp0	ALUSrc	Jump	Branch	MemRd	MemWr	RegWr	MemtoReg
R型	1	1	0	0	0	0	0	0	1	0
lw	0	0	0	1	0	0	1	0	1	1
sw	X	0	0	1	0	0	0	1	0	X
j	X	X	X	X	1	0	0	0	0	X
beq	X	0	1	0	0	1	0	0	0	X

